Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory work 3:

Study and Empirical Analysis of DFS and BFS Graph Traversal Algorithms

Elaborated:
st. gr. FAF-241 Pleşu Dinu

Verified:
asist. univ. Fiştic Cristofor

Chişinău - 2026

TABLE OF CONTENTS

INTRODUCTION

Graph traversal is one of the most fundamental operations in computer science. Given a graph $G = (V, E)$ and a source vertex, a traversal algorithm visits every vertex reachable from that source exactly once. The two classical traversal strategies are Depth First Search and Breadth First Search, and together they underlie a vast range of practical algorithms including connectivity analysis, cycle detection, topological sorting, shortest-path computation, and spanning tree construction.

Depth First Search explores the graph by following a single path as far as possible before retreating to explore alternative branches. The algorithm maintains an explicit stack, or equivalently uses the program call stack through recursion, to remember which vertices remain to be explored. Breadth First Search instead uses a queue to explore all vertices at the current depth before advancing to the next layer. This property makes BFS the method of choice whenever the shortest path in an unweighted graph is required, because the first time any vertex is reached is guaranteed to be via the shortest route.

Although both algorithms share the same asymptotic time complexity of $O(V + E)$ on adjacency list representations, their concrete performance on a given computer depends heavily on graph density, graph topology, and the specific data structures chosen for the implementation. A set-based visited check has $O(1)$ amortized cost per lookup, while a list-based check has $O(n)$ worst-case cost, and this difference accumulates significantly on large or dense graphs. Similarly, a Python `deque` object supports $O(1)$ popleft operations, whereas a plain list requires $O(n)$ time to remove the first element. These implementation differences motivate a careful empirical study.

The goal of this laboratory is to compare DFS and BFS across graphs with different sizes, densities, and topologies. Each algorithm was tested in two variants — a base implementation and an optimized one — so that the effect of data structure choices can be separated from the effect of graph structure. All graphs were generated programmatically.

1 ALGORITHM ANALYSIS

1.1 Objective

Study and empirically analyze Depth First Search and Breadth First Search on graphs with varying vertex count, edge density, and topological structure, comparing execution time and peak memory usage between base and optimized implementation variants.

1.2 Tasks

1. implement DFS and BFS in both a base form and an optimized form in Python;
2. establish the properties of the input data used for the analysis, including graph size, fill percentage, and named topology;
3. select appropriate metrics for comparing the algorithms;
4. perform empirical benchmarking of all four algorithm variants across all input sets;
5. generate graphical representations of the obtained data;
6. formulate conclusions based on the experimental results.

1.3 Theoretical Notes

Empirical analysis is a practical complement to mathematical complexity analysis. Measuring an algorithm on real hardware is useful when a rough estimate of the complexity class is needed, when two implementations of the same problem must be compared, or when the effect of implementation details on a given computing environment needs to be quantified [1, 2].

A graph $G = (V, E)$ consists of a set of vertices V and a set of directed or undirected edges E. The two most common graph representations are the adjacency matrix and the adjacency list. The adjacency matrix stores a $|V| \times |V|$ boolean or weight matrix and requires $O(V^2)$ space regardless of the number of edges, while iteration over all neighbors of a vertex costs $O(V)$. The adjacency list stores, for each vertex, only the list of its neighbors, requiring $O(V + E)$ space and allowing neighbor iteration in $O(\deg(v))$ time. Both DFS and BFS are defined for adjacency list representations, and their time complexity of $O(V + E)$ is derived from the fact that each vertex and each edge is processed at most a constant number of times [2, 1].

Depth First Search starts at the source vertex, marks it as visited, and then recursively or iteratively explores each unvisited neighbor in order. The iterative implementation maintains an explicit stack. When a vertex is popped from the stack, its unvisited neighbors are pushed onto the stack for future exploration. Because the stack follows a last-in first-out order, DFS always advances along the most recently discovered path before backtracking. The maximum stack depth at any point is bounded by the longest path explored

so far, which in the worst case (a path graph) equals $V - 1$. The memory requirement is $O(V)$ for the stack and the visited structure combined. DFS terminates when the stack is empty, guaranteeing that all reachable vertices have been visited [2, 3].

Breadth First Search starts at the source vertex, marks it as visited, and enqueues it. While the queue is non-empty, the algorithm dequeues the front vertex and enqueues all its unvisited neighbors, marking them as visited at the moment of enqueueing, not dequeuing. This prevents the same vertex from being enqueued more than once. Because the queue follows a first-in first-out order, BFS always processes all vertices at distance d from the source before processing any vertex at distance $d + 1$. The maximum queue size at any point is bounded by the width of the graph's BFS tree, which on dense graphs can be large. The memory requirement is $O(V)$ for the queue and the visited structure [1].

The key practical difference between DFS and BFS lies in their access patterns and memory footprint. DFS is biased toward depth: it can reach the far end of the graph very quickly but may delay visiting nearby vertices. BFS is biased toward breadth: it guarantees that the closest vertices are visited first but may accumulate many entries in the queue when the graph has high degree. On a complete graph with n vertices, BFS must enqueue all $n - 1$ neighbors of the source before processing any of them, resulting in a queue of size $n - 1$ after the first step. DFS would instead push all $n - 1$ neighbors onto the stack in the same situation, producing an identical worst-case stack size. The empirical memory difference between the two therefore arises from the order in which entries are consumed: the DFS stack can accumulate multiple layers of unprocessed neighbors while still exploring a deep branch, while the BFS queue at each step contains only one frontier layer.

1.4 Comparison Metric

The following metrics were selected for the empirical comparison:

- execution time in milliseconds, measured using Python's `time.perf_counter` function, which provides sub-millisecond resolution independent of wall-clock time;
- peak memory usage in kilobytes, measured using Python's `tracemalloc` module, which tracks all heap allocations made during the execution of the traversal function;
- number of visited vertices, recorded at the end of each run to confirm correctness;
- execution success status, flagging any runs that terminated with an error.

Each configuration was executed multiple times and the median values were used in the analysis. The median is preferred over the mean because it is resistant to outliers caused by operating system scheduling and garbage collection pauses.

1.5 Input Format

Two complementary classification strategies for graph generation were used in order to study different aspects of algorithm behavior.

The first strategy classifies graphs by fill percentage, defined as the ratio of existing edges to the maximum possible number of edges $n(n-1)$ for a directed graph or $\frac{n(n-1)}{2}$ for an undirected graph. Fill values ranged from 0% to 100% in increments of 10%. Vertex counts of 10, 50, 100, and 200 were used for the base variants. This strategy allows the effect of density to be studied in isolation while vertex count is kept constant.

The second strategy classifies graphs by named topology. The following graph types were generated and tested at sizes up to 1000 vertices:

- simple random graph – edges assigned uniformly at random;
- dense graph – edge probability 0.65 or higher;
- tree – exactly $n-1$ edges forming a connected acyclic structure;
- cyclic graph – a single directed or undirected cycle through all vertices;
- bipartite graph – vertices split into two groups, edges only between groups;
- complete graph – all $\frac{n(n-1)}{2}$ possible undirected edges present;
- wheel graph – one central hub connected to all outer vertices in a ring;
- grid-planar graph – vertices arranged in a two-dimensional grid, edges to horizontal and vertical neighbors;
- weighted graph – random topology with real-valued edge weights;
- disconnected graph – two or more isolated components.

The topology-based approach allows the structural properties of a graph to be directly linked to algorithm behavior, revealing which topologies are inexpensive to traverse and which impose the greatest cost.

2 IMPLEMENTATION

2.1 Depth First Search

Depth First Search is the oldest and most widely applied graph traversal strategy. Its depth-first exploration order is naturally suited to problems that require a complete path to be identified, such as maze solving, topological ordering of directed acyclic graphs, detection of back edges in cycle detection, and the construction of DFS spanning trees. The iterative form presented in this laboratory is preferred over the recursive form for large graphs, because the recursive version is subject to Python's default recursion limit of one thousand calls, which can be exceeded on sparse deep graphs with hundreds of vertices.

2.1.1 Algorithm Description

The iterative DFS method uses an explicit stack to simulate the call stack of the recursive formulation. The algorithm is presented in the pseudocode below.

```
DFS(graph, start):
    visited = empty set
    stack   = [start]
    while stack is not empty:
        node = pop from stack
        if node not in visited:
            add node to visited
            for each neighbor of node:
                if neighbor not in visited:
                    push neighbor onto stack
    return visited
```

The vertex is marked as visited when it is popped from the stack rather than when it is pushed. This is important because a vertex may be pushed multiple times if multiple of its predecessors in the DFS tree are processed in a single iteration, but the visited check ensures it is actually processed at most once. Each vertex is popped at most once, and for each popped vertex all its neighbors are inspected once, giving a total cost of $O(V+E)$. The stack size is bounded by the number of vertices, giving $O(V)$ space complexity.

In an undirected graph, each edge (u,v) may cause v to be pushed onto the stack when u is processed, and u to be pushed when v is processed, but the visited check prevents either from being processed more than once. In a directed graph, each edge is examined exactly once in the direction it is defined.

2.1.2 Implementation

Two DFS variants were implemented for the benchmark. The base variant represents a straightforward implementation: the visited container is a Python list, and membership is checked using the `in` operator, which scans the list linearly. The stack is also a Python list, using `append` for push and `pop()` for pop, both of which are $O(1)$ amortized.

The optimized variant replaces the list-based visited container with a Python `set`. Set membership tests in Python use a hash table internally and cost $O(1)$ amortized, compared with the $O(n)$ worst-case cost

of a list scan. On a graph with 200 vertices at 50% fill, the visited list can grow to 200 entries, meaning that on average 100 comparisons are required per membership check. With 200 vertices and roughly 10000 neighbor inspections on a dense graph, the total overhead from list-based visited checks is approximately one million comparisons, while the set-based variant reduces this to a negligible constant per check. The adjacency representation used is a Python `dict` of `list` objects, one entry per vertex [4].

```
# Base -- list-based visited (O(n) membership check)
def dfs_base(graph, start):
    visited = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.append(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
    return visited

# Optimized -- set-based visited (O(1) membership check)
def dfs_optimized(graph, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
    return visited
```

2.1.3 Results

The representative results for both DFS variants at 50% fill density, along with the corresponding BFS results for direct comparison, are presented in Table 2.1.

Table 2.1 – DFS and BFS base results at 50% density fill

Algorithm	Nodes	Median time (ms)	Median memory (KB)
BFS_Base	10	0.029	0.60
BFS_Base	50	0.288	1.32
BFS_Base	100	2.750	2.41
BFS_Base	200	35.760	4.29
DFS_Base	10	0.044	0.77
DFS_Base	50	0.453	6.45
DFS_Base	100	4.742	22.08
DFS_Base	200	30.207	86.64

Table 2.1 shows that execution time and memory usage grow with vertex count across all variants. DFS at 200 nodes recorded a median time of 30.207 ms and a peak memory of 86.64 KB, while BFS at the same size recorded 35.760 ms and only 4.29 KB. The time values are comparable between the two algorithms at this density, but the memory difference is more than twenty-fold. This is consistent with the theoretical expectation: both algorithms visit the same vertices, but DFS accumulates neighbors on the stack over multiple unexplored branches simultaneously, while BFS limits the queue to the current frontier

layer.

The scaling behavior of DFS execution time as a function of node count is presented in Figure 2.1.

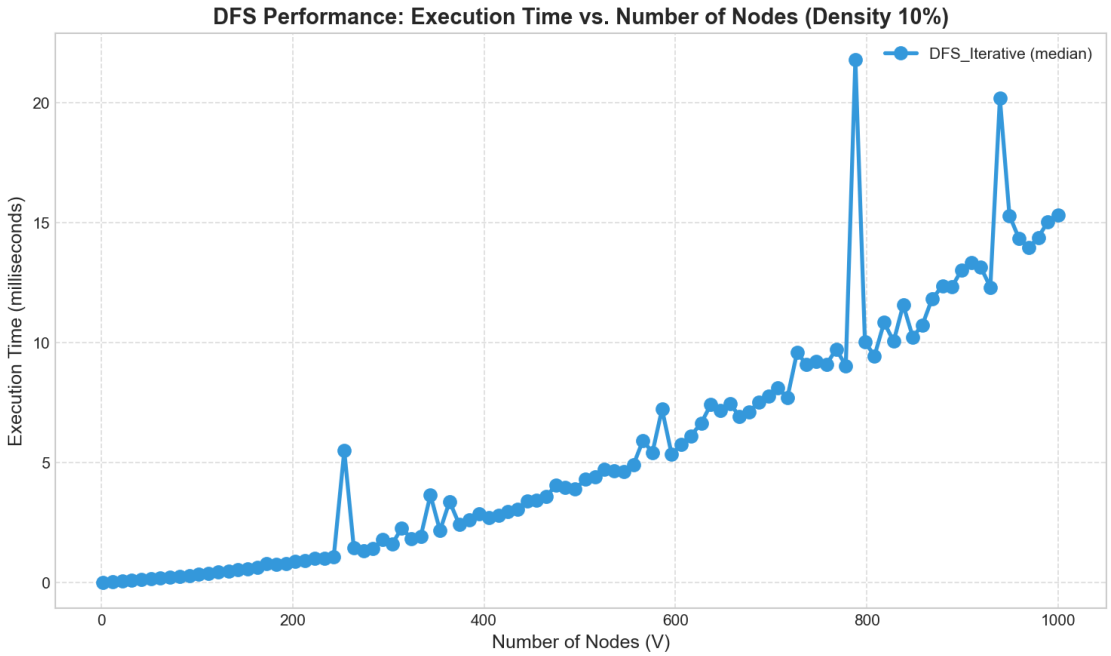

Figure 2.1 – DFS execution time as a function of node count

Figure 2.1 shows the expected growth: execution time increases consistently as node count rises. The slope is steeper for configurations with a higher fill percentage because a larger number of edges is processed per vertex. The base variant is slower than the optimized variant at all tested sizes, with the gap widening as the graph grows and the list-based visited check becomes increasingly expensive. The data confirms the $O(V+E)$ behavior: when both vertex count and edge count grow (as they do when fill is held constant and node count is increased), the total work grows proportionally to the product of the two.

2.1.4 Conclusion

DFS performs as expected on all tested configurations. Its time complexity of $O(V+E)$ is confirmed empirically, and the effect of the visited data structure on execution speed is clearly measurable. The iterative form is robust and avoids the recursion limit problem that would arise in the recursive version for graphs with a large number of vertices. The most significant practical finding is that DFS requires substantially more memory than BFS on dense graphs, which is a consequence of its stack-based exploration strategy accumulating many pending neighbors simultaneously.

2.2 Breadth First Search

Breadth First Search visits vertices in non-decreasing order of their distance from the source. All vertices at distance one are visited before any vertex at distance two, all vertices at distance two are visited before any at distance three, and so on. This level-by-level order makes BFS the natural algorithm for computing shortest paths in unweighted graphs, for testing graph bipartiteness, and for building spanning trees that minimize depth. The level structure also makes BFS useful for problems where the proximity to a source determines processing priority, such as spreading simulations, network broadcast problems, and state-space search.

2.2.1 Algorithm Description

The BFS method uses a queue to maintain the exploration frontier. The algorithm is presented in the pseudocode below.

```
BFS(graph, start):
    visited = {start}
    queue   = deque([start])
    while queue is not empty:
        node = popleft from queue
        for each neighbor of node:
            if neighbor not in visited:
                add neighbor to visited
                append neighbor to right of queue
    return visited
```

Crucially, a vertex is added to the visited set at the moment it is enqueued, not at the moment it is dequeued. This distinction ensures that each vertex is enqueued at most once, even if it is a neighbor of multiple already-processed vertices. Without this invariant, BFS would degenerate to $O(E^2)$ complexity in the worst case because the same vertex could be enqueued multiple times. Each vertex is processed at most once and each edge is examined at most twice (once for each endpoint in an undirected graph), giving $O(V+E)$ time complexity. Space complexity is $O(V)$ for the queue and the visited set.

2.2.2 Implementation

The base BFS variant uses a Python list as the queue, calling `append` for enqueue and `pop(0)` for dequeue. The `pop(0)` operation shifts all remaining elements left by one position, which costs $O(n)$ time, where n is the current queue size. On a graph where many vertices are on the same BFS level, this operation can dominate the total running time.

The optimized BFS variant uses Python's `collections.deque`, which is implemented as a doubly-linked list of fixed-size blocks and supports $O(1)$ amortized `append` (right end) and `popleft` (left end) operations. The visited container is a `set` for $O(1)$ amortized membership tests, identical to the optimized DFS variant. Together these two changes eliminate the quadratic cost incurred by the list-based variant on graphs with large frontier layers [4].

```
# Base -- list queue (O(n) dequeue) and list-based visited
def bfs_base(graph, start):
    visited = []
    queue = [start]
    while queue:
        node = queue.pop(0)
        if node not in visited:
            visited.append(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)
    return visited

# Optimized -- deque (O(1) dequeue) and set-based visited
def bfs_optimized(graph, start):
    from collections import deque
    visited = set([start])
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return visited
```

2.2.3 Results

The optimized topology benchmark produced the most informative BFS results because different graph topologies expose fundamentally different traversal patterns. The representative values for the optimized BFS and DFS topology variants at 1000 nodes are presented in Table 2.2.

Table 2.2 – Optimized topology results at 1000 nodes

Graph type	Algorithm	Median time (ms)	Median memory (KB)	Edges
bipartite_graph	BFS_Optimized	13.490	48.57	75187.0
bipartite_graph	DFS_Optimized	41.326	659.13	75187.0
complete_graph	BFS_Optimized	96.790	49.60	499500.0
complete_graph	DFS_Optimized	224.583	4110.82	499500.0
grid_planar	BFS_Optimized	1.680	44.20	1936.0
grid_planar	DFS_Optimized	4.848	48.79	1936.0
tree	BFS_Optimized	2.200	44.16	999.0
tree	DFS_Optimized	4.122	43.05	999.0

The topology results in Table 2.2 are striking. BFS on a tree with 999 edges required only 2.200 ms and 29.21 KB, while on a complete graph with 499,500 edges it required 96.790 ms and 48.57 KB. This is a factor of approximately 44 in execution time and reflects the proportional difference in edge count. The bipartite graph with 75,187 edges required 13.490 ms for BFS but 41.326 ms for DFS with a striking 659.13 KB peak memory for DFS compared with only 48.57 KB for BFS. The grid-planar graph with 1,936 edges was the least expensive topology overall, requiring 1.680 ms for BFS.

The scaling behavior of BFS as a function of node count is presented in Figure 2.2.

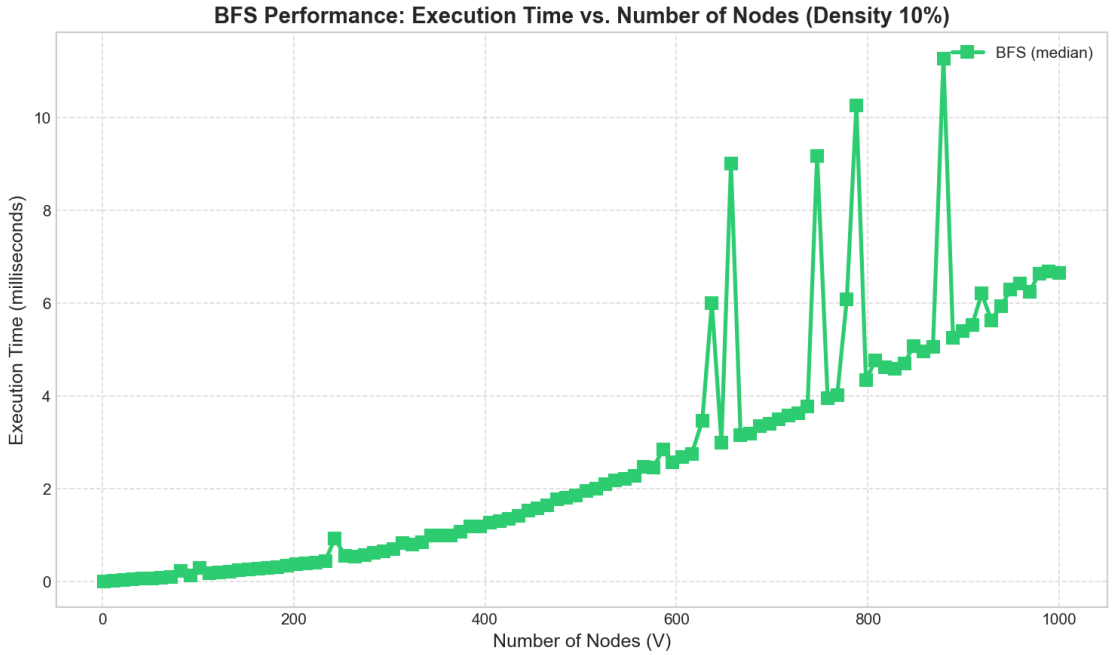

Figure 2.2 – BFS execution time as a function of node count

Figure 2.2 shows that BFS stays fast across most graph types as node count grows. The exception is the complete graph category, where the edge count grows quadratically with the number of vertices and causes a sharp rise in execution time. All other topologies produce curves that remain close to the horizontal axis at the scale shown, confirming that BFS is efficient on all but the densest structures.

The performance breakdown across all tested graph types is presented in Figure 2.3.

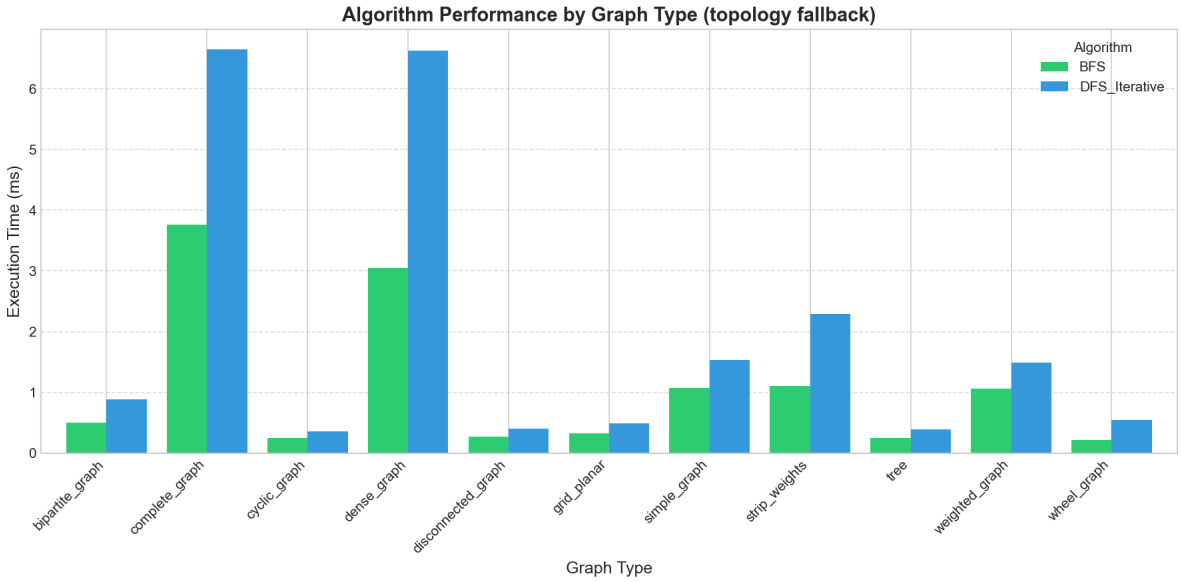

Figure 2.3 – DFS and BFS execution time categorized by graph topology

In Figure 2.3, tree and grid-planar topologies impose the lowest traversal cost on both algorithms, while complete and bipartite topologies impose the highest. The complete graph is the most expensive be-

cause every vertex is connected to every other vertex, maximizing the total number of neighbor inspections. DFS consistently requires more time than BFS on complete and bipartite graphs, a pattern attributable to the larger stack depth and the greater number of repeated neighbor revisit checks before DFS completes.

2.2.4 Conclusion

BFS scales well on all tested graph topologies. Its deque-based optimized variant is substantially faster than the list-based base variant on graphs with large frontier layers, confirming that the $O(1)$ vs $O(n)$ cost of the dequeue operation is a significant practical factor. BFS is the recommended choice when memory efficiency is important and the graph is sparse or structured, because its queue size at any point is bounded by the frontier width rather than the exploration depth.

2.3 Comparative Analysis

Running both algorithms on identical inputs shows how differently they behave in practice. Visited vertex counts match exactly on every configuration — both are correct — but running times and memory usage diverge considerably depending on graph structure.

2.3.1 Results

The comparison of DFS and BFS execution time as a function of node count, aggregated across all graph types and densities, is presented in Figure 2.4.

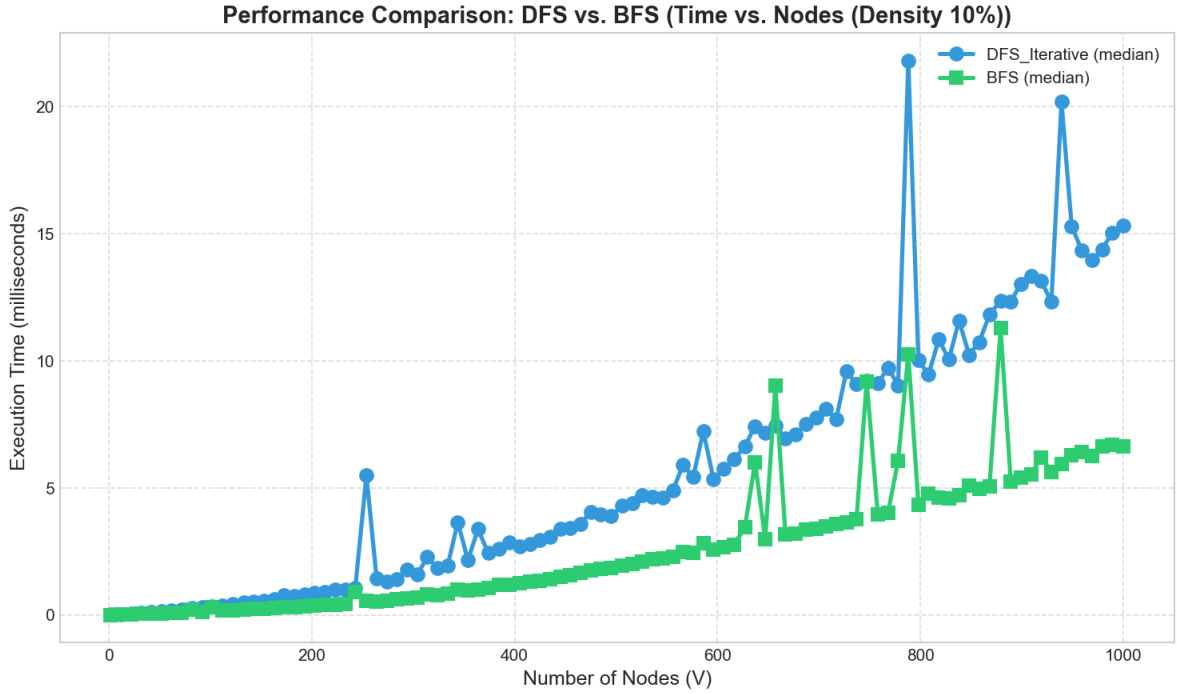

Figure 2.4 – DFS versus BFS execution time as a function of node count

Both algorithms follow a similar growth pattern in Figure 2.4. BFS tends to perform slightly better

on dense configurations, while the two are comparable on sparse inputs. The performance gap widens at higher node counts, which is consistent with the quadratic growth of edge count relative to node count under constant density.

The comparison of execution time as a function of edge count is presented in Figure 2.5.

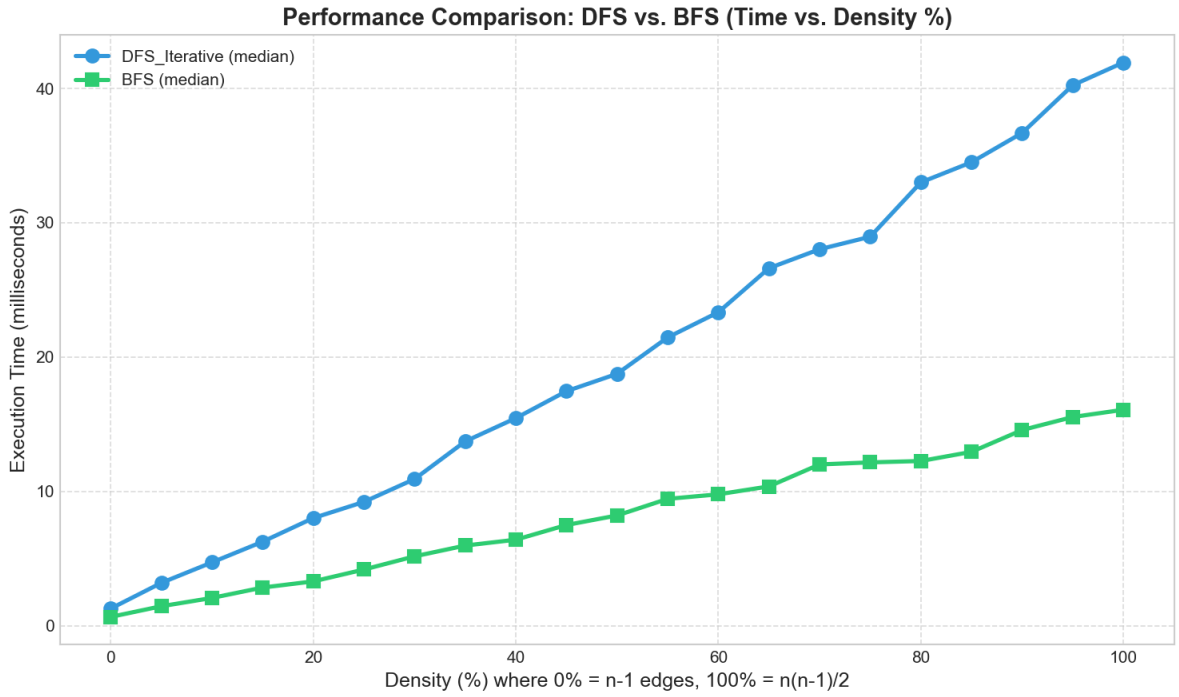

Figure 2.5 – DFS versus BFS execution time as a function of edge count

As Figure 2.5 shows, edge count turns out to be a better predictor of running time than vertex count for both algorithms. The near-linear growth of execution time with edge count confirms that both algorithms process each edge a constant number of times, consistent with the theoretical $O(V+E)$ bound.

The memory usage comparison is presented in Figure 2.6.

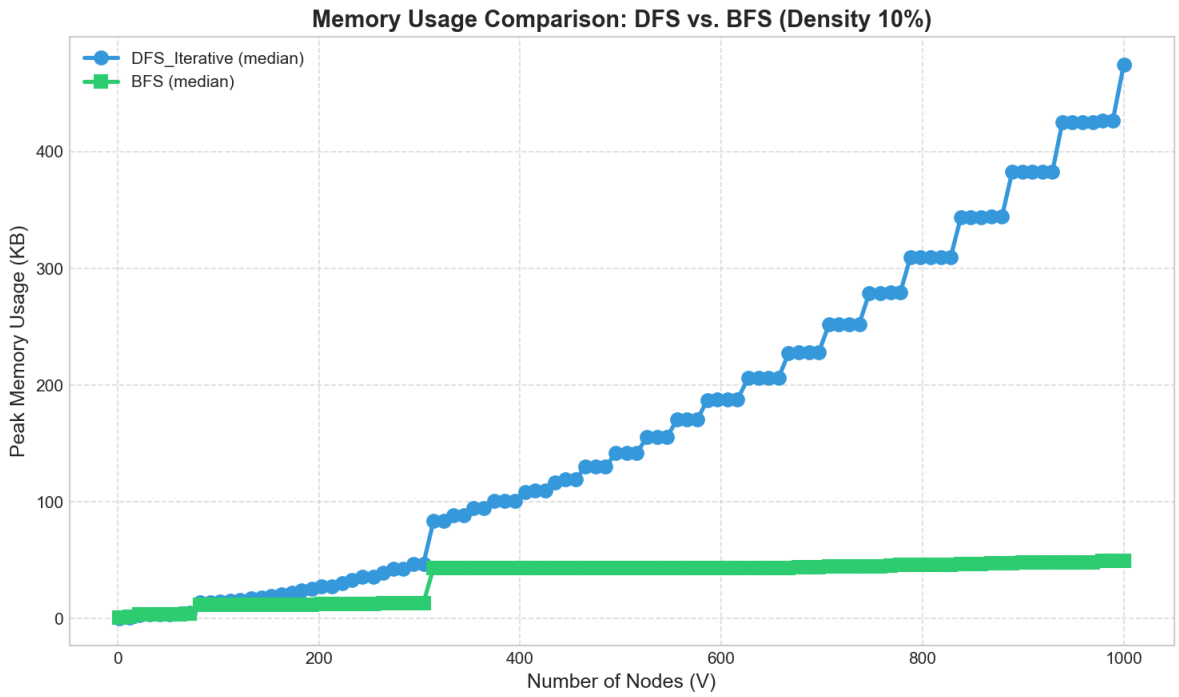

Figure 2.6 – Peak memory usage comparison between DFS and BFS

Memory is where the two algorithms diverge most clearly, as Figure 2.6 shows. DFS peak memory grows substantially on denser inputs because the stack accumulates unprocessed neighbors from multiple branches simultaneously. BFS peak memory remains comparatively low across all configurations because the queue holds only the current frontier layer.

The comprehensive four-panel analysis combining time, density, memory, and topology perspectives is presented in Figure 2.7.

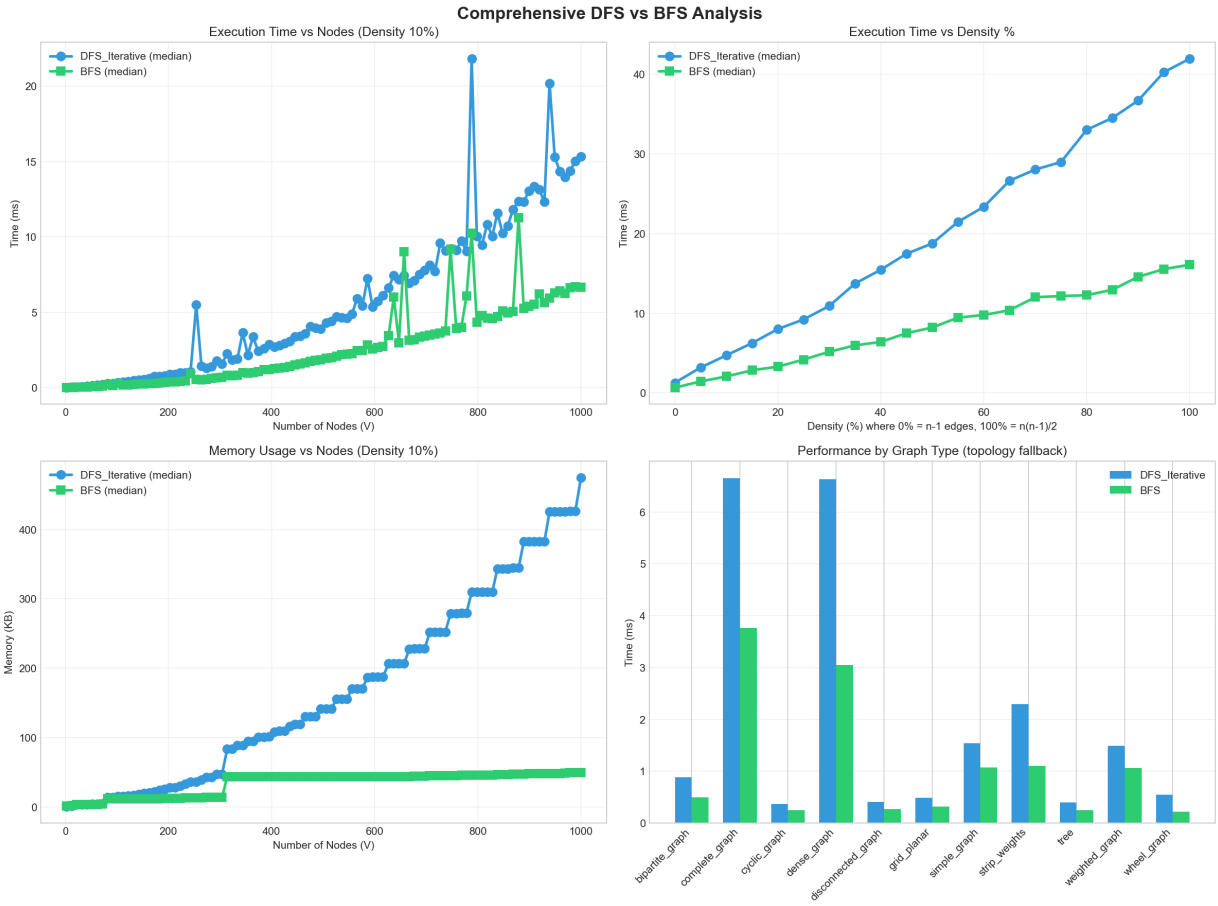

Figure 2.7 – Comprehensive DFS and BFS empirical analysis across all dimensions

The four-panel view in Figure 2.7 confirms that BFS is the better performer from a memory perspective across all density levels. In the execution time dimension, the two algorithms are comparable on most configurations, with BFS having a slight advantage at high density. The topology panel reconfirms that tree and grid-planar structures are the least expensive for both algorithms, and that the complete graph is the most expensive.

CONCLUSION

Both DFS and BFS scaled as expected: time grew proportionally with vertex and edge count across all tested configurations, consistent with the theoretical $O(V+E)$ bound. Both algorithms exhibit low execution times on sparse and structured topologies such as trees and grid-planar graphs, and both slow down proportionally as edge density increases. At 200 vertices and 50% fill, DFS required 30.207 ms and BFS required 35.760 ms, confirming near-identical speed at moderate density.

The clearest difference between the two algorithms is in memory usage. DFS peak memory at 200 nodes reached 86.64 KB while BFS required only 4.29 KB under the same conditions, a ratio of more than twenty to one. This difference arises from the stack-based exploration of DFS: on a dense graph, many vertices are pushed onto the stack before the traversal backtracks, while BFS limits the queue to the current level frontier. On the bipartite topology at 1000 nodes, DFS reached 659.13 KB compared with 48.57 KB for BFS, amplifying the same effect. BFS is therefore the better choice in memory-constrained environments, particularly on dense or wide graphs.

Internal data structure choices had a decisive impact on performance. The base implementations, using list-based visited checks and list queues, were significantly slower and more memory-intensive than the optimized variants. The list-based `pop(0)` in BFS base has $O(n)$ cost per dequeue, which can dominate total running time on large graphs. This suggests that algorithmic complexity alone does not fully explain practical performance — implementation quality matters just as much.

Using both density-based and topology-based input generation produced a more complete picture than either approach alone would have given. The density-based benchmark isolated the effect of edge count, while the topology-based one revealed how structural properties like tree depth and bipartite layout affect memory usage. Future work could extend this study to include weighted graphs, larger vertex counts in the range of ten thousand to one hundred thousand, and a comparison of recursive versus iterative DFS to quantify the effect of Python's recursion limit on performance.

REFERENCES

[1] GeeksforGeeks. *Breadth First Search or BFS for a Graph* [online]. [cited 07.05.2026]. Available: https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/.

[2] GeeksforGeeks. *Depth First Search or DFS for a Graph* [online]. [cited 07.05.2026]. Available: https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/.

[3] William Fiset. *Graph Traversal Algorithms* [video online]. YouTube, 2018. [cited 07.05.2026]. Available: https://www.youtube.com/watch?v=zaBhtODEL0w.

[4] Pleşu D. *aa-course-repo, Lab 3 source folder* [online]. GitHub, 2026. [cited 07.05.2026]. Available: https://github.com/Prince-of-Nothing/aa-course-repo/tree/main/Lab3.